

Treasure Hunt

Time limit: 8 s Memory limit: 256 MiB

You are searching for a long-lost treasure on a massive $N \times N$ field of cells with a magical compass. There are a total of $K \leq 3$ treasure chests hidden at various distinct cells in the field. Your goal is simple: to find them all!

When you place the compass on one of the cells, it will find the shortest paths to a treasure chest using only moves left, up, right, and down (i.e. a closest treasure chest according to Manhattan distance). It will then show the direction of the first move. If multiple valid first moves lead to a closest treasure chest (or several of them), the compass will return all of them. If the cell contains treasure, the compass will indicate that instead.

Whenever you find a treasure chest, you empty the contents, but you cannot remove the chest – it is too heavy. Your compass does not know whether a chest is full or empty. It will point the way to a closest chest, empty or full.

Try to find the location of all treasure chests in the fewest compass queries possible.

Task

This is an interactive task. In each test case (i.e. each execution of your program), your program will have to solve several treasure hunts. You should interact with the grader by using a library provided by the organisers. This library contains the following declarations:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — call this function to start the next treasure hunt. It will set the grid size in the variable *N* and the number of treasures in *K*. If there are no more treasure hunts to be solved in the current run, the function will set *N* and *K* to -1 ; in that case, you should then terminate your program with the exit code 0. Note that you may call this function before finding all the treasures on the current hunt, e.g. if you are trying to only solve for partial points.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — these are constants used in the return values of the *Query* function (see below).
- **int** *Query*(**int** *x*, **int** *y*) — if the cell at coordinates (x, y) contains a treasure, this function returns *TREASURE*. Otherwise it returns the sum of one or more of the constants *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT*, and *DIR_DOWN*, indicating which moves the compass returns when placed on the cell (x, y) . The coordinates *x* and *y* must be integers from 0 to $N - 1$. (Note: in this task, *y*-coordinates increase from top to bottom.)

After *NextHunt* sets $N = K = -1$, your program must not call either *NextHunt* or *Query* again. It must also not call *Query* before the first call to *NextHunt*. If your program fails to observe these constraints, or if it makes more than 1000 queries within a single treasure hunt, or if it provides out-of-range values of *x* and/or *y* when calling *Query*, the library will terminate your program and return a run-time-error (RTE) verdict for the current test case.

A treasure is considered to have been found if you queried the cell containing it at least once. If your program calls *NextHunt* before finding all the treasures of the current hunt, this is not treated as an error, but it will affect your score (more on scoring below).

To access the library, your program should include the header file `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

You can download this header file here: `treasurehuntlib.h`.

To help you in developing your solution, a simple implementation of the library is available here: `treasurehuntlib-public.cpp`. To compile it together with your program, simply add its name as a parameter to the compiler, e.g.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

if `foo.cpp` is the name of the file containing your solution.

The implementation in `treasurehuntlib-public.cpp` supports, in addition to the above declarations, another function called `void InitFromFile(const char *fileName)`, which reads a list of treasure hunts from a file and lets you play the game with those instead of generating its own random treasure hunts. You will find more details inside `treasurehuntlib-public.cpp` itself.

On the evaluation server, a different implementation of the library will be used, so you should not make any assumptions about how exactly the implementation works. You may, however, assume that it does not pollute the global namespace with any declarations other than those listed above (*NextHunt*, *Query*, and the five constants).

Your code must not read from the standard input or write to the standard output, because those will be used by our implementation of the library on the evaluation server to communicate with the rest of the evaluation environment.

Input

Constraints

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Within any single execution of your program, there will be at most 100 000 treasure hunts.
- Within any single treasure hunt, you may make at most 1000 queries.
- The grading system is not adaptive.

Subtasks

- Subtask 1 (10 points) $K = 1$
- Subtask 2 (30 points) $K = 2$
- Subtask 3 (60 points) $K = 3$.

Scoring

A subtask may consist of several test cases (several executions of your program), and each test case may consist of several treasure hunts. For the purposes of scoring, all the treasure hunts of a given subtask are evaluated together, regardless of how they were originally split amongst the test cases. For the i -th treasure hunt, let us denote the grid size by $N_i \times N_i$, the number of treasures by K_i , the number of queries made by your program by Q_i and the number of treasures found by your program by F_i . Let us furthermore denote by S the total number of points available for this subtask. Then the number of points awarded to your program for this subtask is:

- If your program always found all the treasures (i.e. if $F_i = K_i$ for all i), its score depends on $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ points, where } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- If your program did not always find all the treasures (i.e. there exists an i such that $F_i < K_i$), it receives

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ points.}$$

In other words, you get half of the points for finding all the treasures, and the other half for finding them in as few queries as possible. For a perfect score, your solution should find all the treasures in at most $11 \lceil \log_2 N_i \rceil$ queries. Between $11 \lceil \log_2 N_i \rceil$ and $20 \lceil \log_2 N_i \rceil$ queries, the score decreases linearly; and if your solution makes more than $20 \lceil \log_2 N_i \rceil$ queries, it will only get the first half of the points for finding all the treasures. (The symbols $\lceil \cdot \rceil$ mean that the value of $\log_2 N_i$ is rounded up to the nearest integer.)

If the formulas above result in a non-integer number of points for the subtask, it will be rounded to the nearest integer.

If your program has a runtime error or doesn't adhere to the aforementioned protocol in using the library, it will get 0 points for the whole subtask. To receive partial points for finding a subset of the treasures, your program therefore needs to finish the search gracefully by calling *NextHunt*.

Example

Call	Return value
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$